

ARTIFICIAL INTELLIGENCE

AI Agents Explained: What Is a ReAct Loop and How Does It Work?

How agents reason, act, and observe their way to a final answer, one step at a time

Maria Mouschoutzi

July 3, 2026 • 10 min read

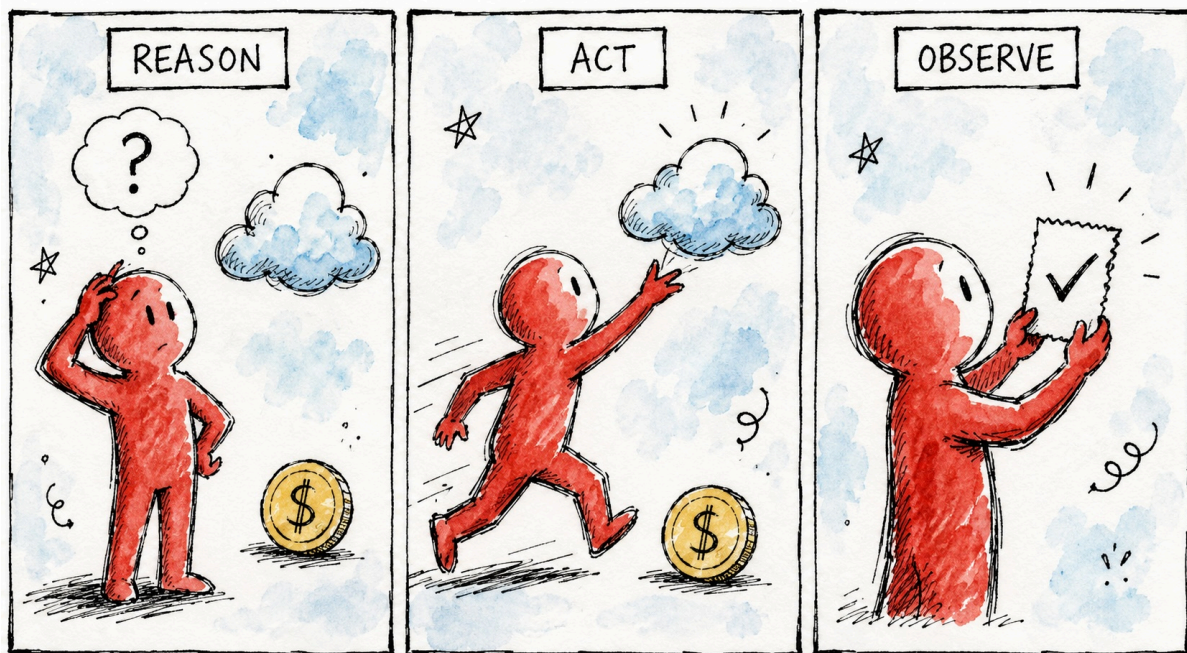


Image created by the author using ChatGPT Images 2.0



In my last post, we talked about **Tool Calling**. Tool Calling is the mechanism that allows an AI model to decide which function needs to be used and



`convert_currency`, or do both at once by calling them in parallel, or neither of them, and just generate text. In other words, the model decides what it needs to do next, we (the rest of the code) execute that decision, pass back the result to the model, and the model ultimately provides an informed answer to the user in text format.

A more advanced version of this loop doesn't stop after just one round of model deciding - code executing - passing back the result - model answering. Instead of generating a response at the end, **the model can use the result of one tool call to decide whether, and which, tool to call next**. As already mentioned at the end of the Tool Calling post, this is a **ReAct loop** (Reason + Act), and is exactly what lets agents handle tasks that can't be solved in a single call.

But what would such a task be? In the previous post's parallel calling example, we asked `What's the weather in Athens and how much is 100 USD in EUR?`, which are two separate things requiring the use of two separate tools to obtain a response, but are also independent from one another. In other words, we can answer those two questions independently, concurrently, without needing any information from the first question in order to reply to the second one.

But what if we ask something like `I bet my friend 100 EUR that it would rain in Athens today. If I won, how many USD is that?` Here, the model won't be able to decide if it needs to call `convert_currency` until it first calls `get_current_weather` and finds out whether it actually rained. Simply put, **the answer to the second question depends entirely on the outcome of the first**. This is precisely the kind of dependency that parallel tool calling can't resolve in one round, and exactly what a ReAct loop is built for.

So, let's take a look!

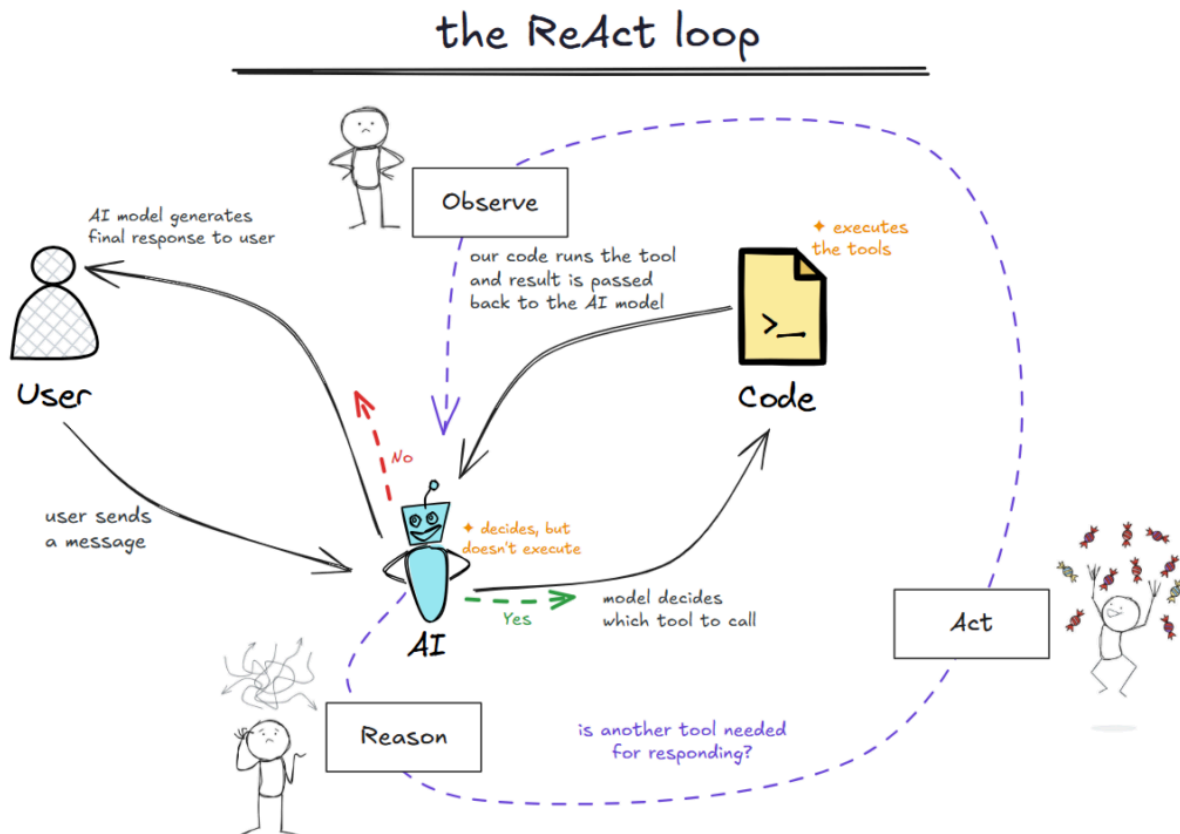


 **DataCream** is a newsletter about AI, data, and tech. If you are interested in these topics, [**subscribe here!**](#)

But what exactly is a ReAct loop?

A **ReAct loop** is just three steps repeated in sequence:

1. **Reason**
2. **Act**
3. **Observe**



At the beginning of the loop, the model **reasons** about what information it already knows and what additional information is missing in order to provide a correct response to the user's query. It then **acts** by calling an appropriate tool with the purpose of obtaining this missing information. Finally, once the respective tool call is executed and its result is passed back to the model, the model **observes** the result (adds the tool's result into its context). Then, it loops back to reasoning again, except this time with this new observation sitting in its context. This loop is repeated until the model evaluates that the available information is enough for answering the user's query, and at this point, it stops calling tools and just responds with text.

But isn't this like the same as the tool calls we already know? Kind of, but not exactly. The part that makes this different from what we covered in the Tool Calling post is the loop itself. In a single tool call, the model asks for something, gets it, and that's the end of the transaction as far as that call is concerned. In the ReAct loop, the conversation remains open, as each new observation becomes new context for the next reasoning step, and the model can change its plan based on what it just learned.

Same Tools, New Trick

To make this concrete, let's go back to the bet example from the intro and think through what the model actually needs to do in order to provide us a reliable answer. The question is: `I bet my friend 100 EUR that it would rain in Athens today. If I won, how many USD is that?` Notice the conditional statement in the middle of it: `if I won`. Whether the model needs to convert any currency at all depends on what the weather call returns. If it rained, the model needs to call `convert_currency` with 100 EUR as an input parameter and give back the converted winnings. If it didn't rain, the bet is lost, `convert_currency` is irrelevant, and the model should just directly return the respective text, without making a second call.

To put it differently, the model genuinely cannot plan its full sequence of tool calls upfront. It has to check the weather first, observe the result, reason about what that result implies for the bet condition, and only then decide whether a second tool call is needed. Unlike the parallel tool calling that worked well for answering `What's the weather in Athens and how much is 100 USD in EUR?`, this question requires a loop.

• • •

The nice thing about a ReAct loop is that it doesn't need new tools. We can still use the same functions, just in a different manner. So we're going to be using `get_current_weather` and `convert_currency` exactly as we built them last time using Open-Meteo for weather and Frankfurter for currency conversion (both still requiring no API key):



```
python
```



```

import requests
import json
from openai import OpenAI

client = OpenAI(api_key="your_api_key")

def get_current_weather(city: str, unit: str = "celsius") -> dict:
    # Step 1: geocode the city name to coordinates
    geo = requests.get(
        "https://geocoding-api.open-meteo.com/v1/search",
        params={"name": city, "count": 1}
    ).json()
    lat = geo["results"][0]["latitude"]
    lon = geo["results"][0]["longitude"]

    # Step 2: fetch current weather
    weather = requests.get(
        "https://api.open-meteo.com/v1/forecast",
        params={
            "latitude": lat,
            "longitude": lon,
            "current": "temperature_2m,precipitation",
            "temperature_unit": unit
        }
    ).json()

    return {
        "city": city,
        "temperature": weather["current"]["temperature_2m"],
        "precipitation_mm": weather["current"]["precipitation"],
        "unit": unit
    }

def convert_currency(amount: float, from_currency: str, to_currency: str) -> dict:
    response = requests.get(
        f"https://api.frankfurter.dev/v2/rate/{from_currency}/{to_currency}"
    ).json()

    rate = response["rate"]
    converted = round(amount * rate, 2)
    return {
        "amount": amount,
        "from_currency": from_currency,
        "to_currency": to_currency,
        "converted_amount": converted,
        "rate": rate
    }

```

Notice one small addition compared to last time: `get_current_weather` now also returns `precipitation_mm`, since that's the field the model needs in order to evaluate the bet condition. Everything else is the same. The `tools` schema is also unchanged from our previous post:

python



```
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_weather",
            "description": "Get the current weather for a given city, including temperature and humidity",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "The name of the city"},
                    "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
                },
                "required": ["city"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "convert_currency",
            "description": "Convert an amount from one currency to another",
            "parameters": {
                "type": "object",
                "properties": {
                    "amount": {"type": "number", "description": "The amount to convert"},
                    "from_currency": {"type": "string", "description": "The source currency"},
                    "to_currency": {"type": "string", "description": "The target currency"}
                },
                "required": ["amount", "from_currency", "to_currency"]
            }
        }
    }
]
```

We also need to define a lookup dictionary that our code will use to dispatch the model's tool choice to the actual Python function:

```
python

available_functions = {
    "get_current_weather": get_current_weather,
    "convert_currency": convert_currency
}
```

This lets us go from a tool name the model gives us back, as a string, to the actual Python function we run. We'll need that mapping in a moment, since this time we don't know in advance how many tool calls we're going to have to resolve, or even whether there will be more than one.

Watching the loop think

Here's the part that's actually new. Instead of making one request and reading off the tool call, **we wrap the whole exchange in a loop**. On each pass, we send the model the full conversation so far, check whether it asked for a tool, run that tool if so, append the result, and go around again. We only stop when the model responds with plain text and no tool calls left to make.

python



```
messages = [
    {
        "role": "user",
        "content": "I bet my friend 100 EUR that it would rain in Athens today. If I won"
    }
]

max_iterations = 5

for i in range(max_iterations):
    print(f"--- Step {i + 1}: Reason ---")

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
        tools=tools
    )

    message = response.choices[0].message
    messages.append(message)

    # If there's no tool call, the model is ready to answer
    if not message.tool_calls:
        print("Final answer:")
        print(message.content)
        break

    # Otherwise, act on every tool call the model requested
    for tool_call in message.tool_calls:
        function_name = tool_call.function.name
        function_args = json.loads(tool_call.function.arguments)

        print(f"--- Step {i + 1}: Act ({function_name}) ---")
        print(f"Calling {function_name} with {function_args}")

        function_response = available_functions[function_name](**function_args)

        print(f"--- Step {i + 1}: Observe ---")
        print(function_response)

        # Feed the observation back in so the next Reason step can use it
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": json.dumps(function_response)
        })
```

Also, notice the `max_iterations` cap preventing a model that decides it needs *"just one more piece of information"* from looping indefinitely. This is of particular importance **because we are paying for every call to the model** within each of those loops.

Ultimately, the resulting observation of the loop is appended as a `role: "tool"` message tied to the specific `tool_call_id`. This allows the model to match each result back to the call that produced it.

And now that we have set up everything, we can finally see the ReAct loop in action.

• • •

So, our bet question can play out two ways depending on what the weather actually is. Let's look at both.

1. If it rained in Athens, our code would print in the terminal something like the following:

markdown



```
--- Step 1: Reason ---
--- Step 1: Act (get_current_weather) ---
Calling get_current_weather with {'city': 'Athens'}
--- Step 1: Observe ---
{'city': 'Athens', 'temperature': 17.4, 'precipitation_mm': 3.2, 'unit': 'celsius'}

--- Step 2: Reason ---
--- Step 2: Act (convert_currency) ---
Calling convert_currency with {'amount': 100, 'from_currency': 'EUR', 'to_currency': 'USD'}
--- Step 2: Observe ---
{'amount': 100, 'from_currency': 'EUR', 'to_currency': 'USD', 'converted_amount': 108.5}

--- Step 3: Reason ---
Final answer:
It did rain in Athens today (3.2mm of precipitation), so you won the bet!
Your 100 EUR comes out to 108.50 USD at today's exchange rate.
```

2. And if it didn't rain in Athens, we would get the following printout:

markdown



```
--- Step 1: Reason ---
--- Step 1: Act (get_current_weather) ---
Calling get_current_weather with {'city': 'Athens'}
```



```
--- Step 1: Observe ---
{'city': 'Athens', 'temperature': 34.1, 'precipitation_mm': 0.0, 'unit': 'celsius'}

--- Step 2: Reason ---
Final answer:
Unfortunately, it did not rain in Athens today, so it looks like you lost the bet.
No currency conversion needed!
```

Look at what happened in the second scenario: the loop ran exactly once. The model observed that `precipitation_mm` was `0.0`, reasoned that the bet condition wasn't met, and stopped without ever calling `convert_currency`. Nobody told it to skip the second tool call, but it rather decided that on its own, based purely on what it observed in the first run of the loop.

This is the major differentiation (at least for this simple scenario) between parallel tool calling and the ReAct loop. In parallel tool calling, we wouldn't be able to exit early from the entire process, and not perform the call `convert_currency`. Instead, in a parallel setup, both tools would have been called upfront, and the model would compose the final response later on. This is of particular importance because remember! **we do pay for every call** to the model. Thus, being able to architecturally narrow down the AI model calls to what we need, without performing unnecessary extra calls, is very substantial.

On my mind

So, when does a ReAct loop actually beat parallel tool calling?

The answer is: whenever the number of tool calls, or the arguments to those calls, can only be determined after seeing an earlier result.

In our bet example, the model can't decide whether to call `convert_currency` at all until `get_current_weather` tells whether it rained. No amount of upfront reasoning resolves that, because the information simply doesn't exist yet within the model's world. We have to step outside of the model's world, pick up external information from the weather API, and add it to the model's context. On the contrary, parallel tool calling assumes the model already knows what it needs before it initiates any tool calls. A ReAct loop doesn't require that assumption: it lets the model discover what it needs as it goes.



In particular, a ReAct loop wins over parallel tool calling the following cases:

1. When one result is a condition for whether another call is needed at all, as in the bet example.
2. When the arguments to a later call depend on the value returned by an earlier one. For example, if the model first had to look up which currency a city uses before it could call `convert_currency` with the right code.
3. When an earlier result comes back unexpectedly, for example, the user may provide a city name that doesn't geocode, or an API returns an error, and the model needs to adapt its plan rather than just report back whatever it got.

Nonetheless, in a straightforward case where all the needed tools and their arguments are obvious from the user's message alone, parallel tool calling is actually the better choice, since in this way we get fewer round-trips, less latency, and the same result.

To me, the most interesting part of moving from parallel tool calling to the ReAct loop is how little code it actually took 😊: a `for` loop, an `if` statement, and a dictionary lookup. Nonetheless, that small amount of code is doing wonders. This ReAct loop, in one form or another, is the actual mechanism behind most of what people mean by an "agent".

✨ Thank you for reading! ✨

• • •

*If you made it this far, **you might find pialgorithms useful** — a platform we've been building that helps teams securely manage organizational knowledge in one place.*

• • •

Loved this post? Join me on 📧 **Substack** and 💼 **LinkedIn**

• • •





Maria Mouschoutzi

Artificial Intelligence

Data Science

Programming

Python

Technology



Write for TDS

Related Articles



This research answered the question: How can machine learning and artificial intelligence help us to unlearn bias?



Subtitle

December 16, 2020 • 7 min read